ACADEMIC RESEARCH PAPER • EMPIRICAL CODE STYLOMETRICS

How Synthetic Minds Write Code: Stylometric Divergence and Cognitive Trajectory Analysis of Large Language Models vs. Human Programmers

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Study Scope: 136 Complete Code Programs (10 Pre-AI Reference Libraries vs 126 LLM Generations)

Abstract

The integration of Large Language Models (LLMs) into software engineering has enabled automated code synthesis. However, code generated by synthetic models exhibits structural, stylometric, and architectural patterns that fundamentally diverge from production code authored by human engineers. This paper presents an empirical comparative study across **136 complete code implementations**, contrasting human-authored standard library routines written prior to the LLM era (2017–2018 code from React, Go stdlib, Redis, Linux, Rust stdlib, PyTorch, and FastHTTP) against frontier synthetic models (*Google Gemini 3.5 Flash*, *OpenAI GPT-5.6 Sol*, and *Anthropic Claude Sonnet 4.6*).

Our quantitative stylometric parser demonstrates that synthetic code exhibits **+306% lines of code (LOC) bloat** (61.0 LOC avg vs. 15.0 LOC human avg), a **+2890% comment-to-code overhead ratio** (28.9% synthetic vs. 0.00% human reference core), a 5.1x density in explicit type annotations, and a 2.3x expansion in micro-helper function fragmentation. We formalize the cognitive divergence between human engineers (who operate under hardware-first, memory-aligned, and state-invariant models) and synthetic models (which operate under statistical token trajectories derived from pedagogical textbook corpora). Finally, we categorize seven systemic architectural anti-patterns in synthetic code—including $O(N^2)$ algorithmic complexity regressions, un-tracked asynchronous timer handle leaks, SIMD auto-vectorization degradation, and prototype pollution vulnerability risks.

1. Introduction

Automated code generation has transitioned from simple template completion to full algorithmic synthesis powered by transformer-based Large Language Models (LLMs). While models demonstrate high pass rates on standard coding benchmarks, code quality in production systems depends on metrics beyond functional correctness: memory alignment, cache locality, zero-allocation micro-optimizations, control flow readability, and domain-specific state invariant enforcement.

When LLMs write code, they do not possess a physical execution model of the underlying CPU or operating system kernel. Instead, they sample token probability distributions shaped by public code repositories, online tutorials, and enterprise software frameworks. This statistical sampling produces a distinct visual and structural signature—what the engineering community colloquially observes as "synthetic code."

2. Methodology & Dataset Composition

To establish an uncontaminated human benchmark, we extracted 10 reference functions directly from open-source repositories authored between 2017 and 2018 (prior to the public availability of modern code LLMs): React 16 Fiber Scheduler (Andrew Clark / Dan Abramov), React 16 shallowEqual, Go 1.10 strings.Builder (Russ Cox / Brad Fitzpatrick), Rust slice::rotate_left, PyTorch 1.0 clamp_out, Redis 5.0 raxInsert (antirez), TypeScript 3.0 isIdentifierStart (Anders Hejlsberg), Linux Kernel 4.14 eBPF htab_map_lookup_elem, FastHTTP caseInsensitiveCompare, and Rust Vec::retain.

For each benchmark task and prompt, we queried three frontier LLM architectures via stateless OpenRouter API invocations: Google Gemini 3.5 Flash, OpenAI GPT-5.6 Sol, and Anthropic Claude Sonnet 4.6.

3. Quantitative Stylometric Results

Stylometric Metric	Human Pre-AI Code	Gemini 3.5 Flash	GPT-5.6 Sol	Claude Sonnet 4.6	Synthetic Average
Average Lines of Code (LOC)	15.0 LOC	55.4 LOC	54.2 LOC	73.5 LOC	**61.0 LOC (+306% Bloat)**
Comment & Docstring Ratio	0.00%	49.48%	0.88%	36.35%	**28.90% (+2890% Overhead)**
Explicit Type Annotations	1.50 / file	8.20 / file	7.90 / file	11.40 / file	**9.17 / file (+511% Density)**
Helper Method Count	1.10 / file	2.40 / file	2.10 / file	3.00 / file	**2.50 / file (2.27x Expansion)**
Return Statements / File	1.70 / file	3.10 / file	2.80 / file	4.37 / file	**3.42 / file (2.01x Branching)**
Vertical Whitespace Ratio	7.33%	16.20%	19.52%	15.60%	**17.11% (2.33x Spacing)**

4. Cognitive Trajectory Divergence: Human vs. Synthetic Mindset

Human Engineering Paradigm: Hardware Alignment & State Invariants — Human software engineers write code under physical machine constraints: CPU bitwise register efficiency (writing `(index - 1) >>> 1` for binary min-heaps), hidden structural safety (writing `b.addr != b` struct copy guards in Go), and single-pass flat execution flow.

Synthetic Engineering Paradigm: Pedagogical Abstraction & Token Trajectories — Synthetic LLM code exhibits a fundamental structural bias towards textbook modularity: micro-helper over-fragmentation (+306% LOC expansion), mathematical abstraction truncation (using `Math.floor((i - 1) / 2)` float ops), and trivial echo commenting parroting syntax line-by-line.

5. The 7 Systemic Architectural Patterns & Blindspots

- **Pattern 1: Micro-Fragmentation & Abstraction Overhead** — Decomposing 15-line algorithms into class hierarchies and 3+ micro-helpers, inflating code by +306%.
- **Pattern 2: Contextual Invariant Blindness** — Omitting struct copy checks, calling `obj.hasOwnProperty()` on `Object.create(null)` objects, or using `===` instead of `Object.is`.
- **Pattern 3: The Trivial Echo Comment Reflex** — 28.9% comment density parroting trivial syntax instead of documenting domain intent.
- **Pattern 4: Closure Overhead in Hot-Paths** — Generating functional iterators (`array.every()`) in V8 hot loops instead of indexed `for` loops.
- **Pattern 5: Algorithmic Complexity Regressions** — Regressing vector filtering from $O(N)$ write-head swaps to $O(N^2)$ via `vec.remove(i)` calls in loops.
- **Pattern 6: Asynchronous State & Timer Handle Leakage** — Spawning un-tracked `setTimeout()` handles without `clearTimeout()`, and mutating subscriber arrays live during event dispatch.
- **Pattern 7: Vectorization & Hardware Compiler Sub-Optimality** — Using `std::min(std::max(...))` template calls whose NaN handling logic breaks GCC/Clang SIMD auto-vectorization passes.

6. Model Stylometric Profiles

Google Gemini 3.5 Flash: Hyper-pedagogical, maximum comment density (49.5%), extensive JSDoc wrappers, Python `__slots__` usage.
OpenAI GPT-5.6 Sol: Enterprise minimal, zero comment noise (0.88%), monolithic routines, coarse-grained global mutex locks.
Anthropic Claude Sonnet 4.6: Comprehensive signature, highest code bloat (73.5 LOC avg), embedded unit test harnesses.

7. Conclusion

Synthetic code generated by Large Language Models possesses a distinct stylometric fingerprint shaped by statistical probability sampling over tutorial code corpora. Understanding these empirical patterns provides

critical insights for static code analysis, software maintenance, and automated LLM code evaluation.